START FROM SCRATCH / HYPER-PACED LEARNING

Week 6: Make the system observable

Turn the SRE Agent from a black box into a system the team can trace, evaluate, and improve.

WHAT YOU ALREADY HAVE

An agent with tools and specialists

The SRE Assistant can retrieve runbooks, call tools, and coordinate specialist work. Now the team needs evidence of what happened across each request.

THIS WEEK'S PLAN

Trace the full request journey

Instrument the model, retrieval, agent decisions, handoffs, tools, approvals, and final response. Add a small set of useful measures and testable quality checks.

RED FLAGS / TESTING

Make failures diagnosable

Test slow or failed tools, missing retrieval, repeated calls, bad handoffs, high token use, and incorrect outcomes. Protect sensitive prompts and retrieved content in logs; observability must not become a data leak.

EXPECTED OUTCOME

A traceable, testable SRE system

Demonstrate one end-to-end trace, a small dashboard or report, and a repeatable evaluation set that helps explain whether the system worked and where it failed.

If the team cannot see how a result was produced, it cannot reliably debug or improve it.

Team rhythm: prepare independently, align with your team and lead, build together, then share. Each team has 15 minutes: 5 minutes to demo and 10 minutes for discussion and feedback.

PREPARE BEFORE YOU BUILD

See the request, not just the final answer

Learn how traces and evaluation complement ordinary application monitoring. Capture enough context to diagnose behavior while deliberately managing sensitive content.

01 **Generative AI semantic conventions**
OpenTelemetry
Review the model, retrieval, and tool-call attributes. Notice the warning that message and query content may include sensitive or personal information.
Review GenAI conventions

02 **Demystifying evals for AI agents**
Anthropic Engineering
Focus on how multi-turn agent runs are evaluated. Think about a small set of incident examples, expected behaviors, and observable intermediate steps.
Read the evaluation guide

Bring these questions to your team

- What should one trace show from the user's request through retrieval, model calls, tools, and final response?
- Which measures help explain reliability, latency, and cost, and which checks assess answer quality?
- What prompt, runbook, or user data should be excluded, redacted, or access-controlled in telemetry?

Do not confuse observability with collecting everything. Decide what to capture, who may see it, how long it is kept, and how the signals help a person take action.

FOLLOW ONE REQUEST END TO END

A useful observability loop

Connect application behavior to operational signals and evaluation results. The goal is to answer: what happened, why did it happen, how often, and what should we improve?

Trace	Measure	Improve
Correlate one request across model calls, retrieval, agent handoffs, tool calls, approvals, and response.	Track latency, errors, retries, token or cost use, tool success, and outcome quality against a baseline.	Use traces and a repeatable evaluation set to find a failure, make one change, and check for regressions.

Start with signals tied to an operator question

Signal	Example question	Possible view
Reliability	Did the model, retrieval, or tool fail?	Error rate and failures by component
Performance	Where did this incident investigation spend time?	End-to-end latency plus per-step spans
Cost and usage	Which flow or tool path consumes the most?	Requests, tokens, calls, and cost by run
Quality and safety	Was the answer grounded and did it stop or escalate correctly?	Evaluation pass rate, evidence, and approval events

Treat telemetry as sensitive data

Prefer metadata and structured spans over indiscriminate prompt capture. Redact or omit secrets and personal data, restrict access, set retention intentionally, and make trace IDs available to the people who investigate failures.

COME READY TO DESIGN

Your Observability Canvas

Agree on the operator questions first. Then decide which events, attributes, checks, and safeguards answer them.

1. Run identity How can each request and its child steps be correlated?	**2. Trace steps** Which model, retrieval, agent, and tool spans matter?
3. Measures Which reliability, latency, usage, and quality measures matter?	**4. Evaluation set** Which incidents and expected behaviors form a repeatable test?
5. Failure path How will timeouts, retries, tool errors, and escalation appear?	**6. Privacy** What is redacted, restricted, and retained?
7. Owner Who reviews a signal and what action can they take?	**8. Demo** Which trace shows both the successful and failed paths?

Show the learning

- Open a trace and explain the request from start to finish, including one intermediate decision.
- Show one injected fault or evaluation failure and locate its cause in the trace.
- Name one signal that would trigger an operational action, not just populate a dashboard.

Make the system explainable to its operators. Show the trace, show the failure, and show how the team would improve it.